

Patient Monitoring Service (PMS)

Part 2: Architecture evaluation

Description of the initial architecture

Contents

1	QAS Decisions	5
1.1	P2: Risk estimation by clinical models	5
1.2	M2: New or optimized clinical models	7
A	Client-Server View	10
B	Decomposition View	11
C	Deployment view	13
D	Process View	15
E	Element catalog	19
E.1	Components	19
E.1.1	ClinicalJobCreator	19
E.1.2	ClinicalModelCache	19
E.1.3	ClinicalModelDB	19
E.1.4	eHealth Platform	19
E.1.5	HIS	20
E.1.6	KVStorageService	20
E.1.7	MLaaS	20
E.1.8	MLModelManager	20
E.1.9	OtherFunctionality	20
E.1.10	RiskEstimationCombiner	21
E.1.11	RiskEstimationProcessor	21
E.1.12	RiskEstimationScheduler	21
E.2	Modules	21
E.2.1	JobMLModelSelector	21
E.2.2	JobProcessor	22
E.2.3	MLaaSLibrary	22
E.2.4	MLLibrary	22
E.2.5	MLModelRepository	22
E.2.6	MLModelRetrieval	22
E.2.7	MLModelStorageManager	23
E.2.8	MLModelUpdateProcessor	23
E.2.9	ModelUpdater	23
E.2.10	SensorDataRiskAssessment	23
E.3	Interfaces	23
E.3.1	ApplyClinicalModel	23
E.3.2	ClinicalJobMgmt	24
E.3.3	ClinicalModelCacheMgmt	24
E.3.4	ClinicalModelStorage	24
E.3.5	FetchClinicalModels	24
E.3.6	FetchJobs	25
E.3.7	FetchMLModel	25
E.3.8	healthAPI	25
E.3.9	JobMgmt	26
E.3.10	KVStore	27
E.3.11	LaunchRiskEstimation	27
E.3.12	MLaaSAPI	27
E.3.13	MLaaSService	28
E.3.14	MLAPI	28
E.3.15	MLModelMgmt	29

E.3.16	MLModelStorage	29
E.3.17	MLModelSync	29
E.3.18	MLModelUpdateMgmt	30
E.3.19	OtherDataMgmt	30
E.3.20	PatientRecordMgmt	30
E.3.21	Results	30
E.3.22	SensorDataAnalysis	31
E.3.23	SensorDataMgmt	31
E.4	Nodes	31
E.4.1	eHealthPlatform	31
E.4.2	MLaaS Conn (Pilot Deployment)	31
E.4.3	MLaaS Service (Dev Deployment)	31
E.4.4	MLaaS Service (Pilot Deployment)	32
E.4.5	MLaaSCloud	32
E.4.6	MLaaSConnectorNode	32
E.4.7	RiskEstimationManagementNode (Pilot Deployment)	32
E.4.8	RiskEstimationManagmentNode (Dev Deployment)	32
E.4.9	RiskEstimationMgmtNode	32
E.4.10	RiskEstimationProcessorNode	32
E.4.11	RiskEstimationProcessorNode (Dev Deployment)	32
E.4.12	RiskEstimationProcessorNode (Pilot Deployment)	32
E.4.13	TODO Node (Pilot Deployment)	32
E.4.14	TODONode	33
E.5	Exceptions	33
E.6	Data types	33
E.7	Unresolved issues	38

Requirements

Note: The full requirements are included here only for the sake of completeness. Your report should not include the requirement as such but the rationale should include the relevant information from the requirements, e.g., any deadlines that have to be achieved.

P2: Risk estimation by clinical models (Medium importance)

As clinical models are typically highly complex and computationally intensive, it might occur that new information (sensor data readings) arrive at the PMS in a faster rate than they can be processed by the risk estimation algorithms/clinical models. The throughput of the (sub-)system responsible for performing risk estimation (a process that is triggered by sensor data reading updates) and issuing notifications for a specific patient is crucial. Especially when the PMS is scaled to a larger patient base, it is realistic that multiple risk estimation calculations are started in parallel, which increases the pressure on this (sub-)system.

- **Source:** new sensor data coming in
- **Stimulus:** patient risk estimation by applying clinical models
- **Artifact:** the (sub-)system responsible for performing risk estimation and issuing notifications
- **Environment:** normal modus
- **Response:**
 1. In normal modus, the subsystem processes the incoming information updates in a first-in, first-out order
 2. In overload modus,
 - the subsystem changes the processing order by prioritization according to the patient's risk level: risk level estimations for patients with a high level are performed before risk level estimation for patients with a medium and low risk level
 - Additionally, the load is balanced over multiple instances
- **Response measure:**
 - In normal modus, the system goes into “overload modus” when the throughput > 20 risk estimations/minute,.
 - In overload modus, there is no starvation of risk estimation jobs for patients with a low risk level: any risk level estimation job must be initiated before a hard 10 minute deadline after arrival into the queue.

M2: New or optimized clinical models (High importance)

Research (result of performing data mining activities on the fine-grained operational data kept by the PMS) has shown that one of the clinical models currently in use for risk estimation can be improved. Therefore, it should be easy to add or update the clinical models to versions that are more efficient (computational-wise) and lead to more correct risk estimations.

- **Source:** us, the PMS
- **Stimulus:** want to optimize risk estimation, and by extension the efficacy of our system
- **Artifact:** the subsystem responsible for risk estimation
- **Environment:** at run time
- **Response:**
 1. The new or updated clinical risk model is deployed at run time (in the running PMS).
 2. This update does not affect currently ongoing risk calculation, even if an existing clinical model is replaced with a new implementation.
 3. The modification only affects the subsystem responsible for risk estimation, but does not lead to down time of this subsystem, other subsystems, or the entire PMS.
- **Response measure:**
 - Deployment or installation of the new clinical model does not take longer than 30 minutes. The updated or new clinical model can be used immediately.

1. QAS Decisions

1.1 P2: Risk estimation by clinical models

Key Decisions

- (AD1) Parallelized and concurrent execution of multiple `RiskEstimationProcessor` instances, on multiple nodes and multiple times on each node
- (AD2) `RiskEstimationScheduler` queues new risk estimation **ClinicalModelJobs**, monitors the throughput and schedules them

Architectural Decision 1: Parallel and concurrent risk estimation

P2 requires the execution of risk estimations to handle an increasing input load. To achieve this, multiple instances of the `RiskEstimationProcessor` are executed concurrently, allowing different risk estimations to be processed in parallel. Each processing node (`RiskEstimationProcessorNode`) asynchronously retrieves risk estimation jobs from a shared queue. These jobs can be treated as independent *commands* (an instance of the **Command** pattern), as risk estimations do not read or modify shared state. Consequently, no concurrency control mechanisms are required.

Parallelism is introduced at two complementary levels. First, multiple processor instances are deployed across **multiple physical nodes**, enabling horizontal scaling of throughput. Second, each processing node may host **multiple concurrent processor instances**, allowing the node to overlap computation with I/O. This is particularly effective because risk estimation typically involves retrieving substantial amounts of data (e.g. monitoring history, current patient status, and patient records), causing processors to block frequently on I/O.

By exploiting both inter-node and intra-node concurrency, the architecture maximizes resource utilization and improves throughput under load. However, care must be taken not to over-provision processor instances on a single node. Excessive concurrency can lead to contention for CPU, memory, or temporary local storage, increasing individual job latency and ultimately degrading overall throughput.

Employed tactics and patterns

Queuing and scheduling of commands, concurrency on multiple physical nodes, concurrency on a single node.

Sensitivity points

The above design allows the system to scale risk estimation throughput with increasing load, while avoiding unnecessary synchronization overhead. Concurrency exploits I/O wait times and avoids idle resources, which is essential given the computationally intensive nature of clinical models. This decision introduces a key *sensitivity point* with respect to performance and scalability: the end-to-end latency and throughput of a risk estimation job are highly sensitive to (i) the cost of per-job data access (clinical models and configurations), and (ii) the locality of that data relative to the processor executing the job. Repeated remote lookups on the critical path would directly undermine the throughput guarantees of *P2*.

Relation to other architectural decisions

The scheduler must cooperate closely with the model deployment and caching mechanisms introduced by *M2*: New or optimized clinical models. Scheduling decisions rely on timely access to job metadata, while model updates may trigger cache invalidation and refresh activity. If these activities overlap without coordination, communication between the scheduler and processor nodes risks becoming a bottleneck.

Trade-off justification

Centralized scheduling simplifies global overload detection and deadline enforcement, but introduces a single, performance-critical control point. This trade-off is acceptable because it enables deterministic prioritization

behavior, which is essential for clinical safety. The architecture trades distribution simplicity for predictable quality enforcement.

Considered alternatives

Caching input data. Depending on the exact risk estimation model used, a risk estimation can require large amounts of input data, possibly from multiple databases and possibly from external databases. Our current design incorporates caching of the clinical models and their configurations (e.g., `ClinicalModelCache`). However, it does not yet incorporate solutions to improve the performance of fetching the required historical data in order to avoid this extra complexity. Should this pose a problem in the future, the architecture can be extended. For example, in order to speed up the complete risk estimation process, data can be fetched and cached asynchronously when a risk estimation job enters the queue instead. As another example, external database with possibly large response times could asynchronously be cached or duplicated locally. Notice that depending on the consistency requirements of these data, race conditions could complicate both examples.

While input-data caching could reduce latency, it introduces extra cache consistency and invalidation mechanisms across many concurrent processors. Under P2 overload conditions this control traffic and coordination can become a bottleneck and may undermine deadline guarantees, while stale caches risk clinically incorrect or non-reproducible outcomes. Therefore we prioritize the simpler, predictable critical path (model/config caching via M2) and only add input caching if measurements show data access dominates job time.

Dynamic scaling. The introduced replication allows the PMS to support at least 20 risk estimations per minute by scaling out the number of processing nodes. The required number of nodes can be estimated using statistical methods on the expected arrival rate of sensor data for the expected number of monitored patients. However, P2 also imposes a hard deadline of 10 minutes for the initialization of each job. While the statistical estimation of the number of processing nodes allows to minimize the chances of failing to keep this deadline, a fixed number of nodes can strictly speaking not handle any peak load. To achieve this, the architecture could be extended with support to dynamically scale out the number of processor nodes for peak loads.

Dynamic scaling can address peaks, but provisioning delays are not bounded and may exceed P2’s time windows (including the 10-minute initiation deadline). It also introduces additional operational dependencies and failure modes exactly in overload situations, reducing predictability and making clinical timing guarantees harder to validate. Hence we rely on sized replication as the baseline and treat autoscaling as an optional extension where bounded startup times can be guaranteed.

Architectural Decision 2: Centralized scheduling and overload mode

To satisfy P2, the system must detect overload conditions and adapt its scheduling behavior accordingly. This responsibility is centralized in the `RiskEstimationScheduler`, which continuously monitors the arrival rate and processing throughput of risk estimation jobs.

Under normal operating conditions, jobs are scheduled in **first-in/first-out** order. When the observed throughput exceeds the threshold defined by P2 (i.e., more than 20 risk estimations per minute), the scheduler switches the system into *overload modus*. In this mode, jobs are scheduled based on patient risk level using **dynamic priority scheduling**, implemented as **earliest deadline first**.

This centralized approach allows the system to enforce global prioritization and deadline guarantees consistently across all processing nodes. By adapting scheduling behavior at runtime, the architecture ensures that high-risk patients are handled preferentially while still preventing starvation of lower-risk jobs.

Employed tactics and patterns

Dynamic priority scheduling, earliest deadline first, first-in/first-out.

Sensitivity points

Deadlines are assigned to the risk estimations based on the level of the corresponding patients according to P2. Scheduling based on deadlines avoids starvation of lower risk level estimations. The resulting deadlines are shown in Table 1.1. Consequently, the system’s ability to meet P2 deadlines is highly sensitive to the responsiveness and availability of the scheduler component. If scheduling decisions or metadata access are delayed, overload detection and prioritization become ineffective.

Risk level	Deadline (minutes)
high	2
medium	5
low	8

Table 1.1: Deadlines for the initiation of risk estimations per risk level in overload modus.

Relation to other architectural decisions

The scheduler must cooperate closely with the model deployment and caching mechanisms introduced by M2: New or optimized clinical models. Scheduling decisions rely on timely access to job metadata, while model updates may trigger cache invalidation and refresh activity. If these activities overlap without coordination, communication between the scheduler and processor nodes risks becoming a bottleneck.

Trade-off justification

Centralized scheduling simplifies global overload detection and deadline enforcement, but introduces a single, performance-critical control point. This trade-off is acceptable because it enables deterministic prioritization behavior, which is essential for clinical safety. The architecture trades distribution simplicity for predictable quality enforcement.

Considered alternatives

Always scheduling approach. The current scheduling approach implements first-in-first-out in normal modus and switches to priority scheduling (based on deadlines) in overload modus. An alternative design would involve scheduling in all cases at the basis of the inherent deadline or priority of each risk estimation job, but this was decided against because such an approach increases the risk of starvation and incurs the constant extra performance cost of fetching or calculating deadlines.

Always-on priority scheduling would require computing/fetching deadlines for every job, adding steady overhead on the scheduler’s critical path. That overhead directly reduces responsiveness for overload detection and global coordination, which P2 depends on to enforce deadlines safely. We therefore keep FIFO in normal mode and only pay the priority cost when overload is detected.

1.2 M2: New or optimized clinical models

Key Decisions

- (AD1) Clinical models are stored in a separate database (ClinicalModelDB), which is append-only. Fetching models and configurations happens at run time and on demand
- (AD2) Models and configurations are cached in the ClinicalModelCache with explicit cache invalidation

Employed tactics and patterns

Defer binding time using database storage, caching, explicit invalidation.

Architectural Decision 1: Run-time deployment of clinical models

This decision enables the deployment and evolution of clinical risk models at run time without requiring changes to application code. Clinical models (**ClinicalModels**) are stored in a dedicated, append-only database (ClinicalModelDB) and are fetched and loaded on demand during execution, together with their patient-specific configurations.

This design allows new clinical models to be introduced and existing ones to be updated dynamically. A particularly important source of change is the set of machine learning models (**MLModel**) referenced by clinical models. These ML models are retrieved and synchronized at run time (Figure D.3), making them immediately available as soon as a clinical model refers to them. Individual ML models may also be updated independently to improve predictive capability, without requiring modifications to the clinical model itself (Figure D.4).

From an architectural perspective, this decision impacts both data storage and the interfaces exposed for creating or modifying clinical models. To prevent unintended side effects on other subsystems, clinical

models are stored separately from operational data such as sensor data. The design assumes that new models operate within the same technological environment as existing ones and that clinical risk estimation remains self-contained. These assumptions are enforced through the required interfaces of `SensorDataRiskAssessment`, which constrain new models to file I/O and external ML library usage only.

Sensitivity points

Run-time deployment allows optimization and improvement of clinical models without downtime or disruption of ongoing operation. This introduces a *sensitivity point* related to correctness and consistency: risk estimation correctness is sensitive to whether a running job observes a stable model version throughout its execution. Any mid-execution change would compromise clinical validity.

Relation to other architectural decisions

This decision interacts directly with P2: Risk estimation by clinical models’s parallel processing and scheduling. Because many jobs execute concurrently across multiple nodes, the architecture must ensure that model updates do not affect in-flight jobs, while still becoming visible to newly started jobs. This requirement drives the need for explicit caching and version isolation mechanisms. Additionally, the changes more heavyweight models may introduce additional performance penalties that have to be taken into account to ensure that the P2 performance deadlines (as also outlined in table 1.1) are met.

Trade-off justification

The architecture prioritizes *run-time modifiability and availability* over instantaneous global consistency. By avoiding in-place updates and using append-only storage, the system guarantees stability of ongoing computations at the cost of temporary version divergence. This trade-off is acceptable because clinical correctness per job is preserved.

Considered alternatives

Dedicated workers for different clinical models. An alternative for storing the **ClinicalModels** in the `ClinicalModelDB` is to create dedicated workers for the calculating the results for the different risk models. These dedicated workers provide a performance advantage as they enable the creation of optimized implementations for every clinical model. Furthermore, the dedicated workers do not require having a common shared set of APIs to work with these clinical models. However, there are several important downsides to this alternative solution. The use of custom workers introduces considerable complication in the management of the `RiskEstimationProcessor` infrastructure, as the infrastructure has to be managed and monitored to ensure that there is an appropriate amount of workers available for every type of clinical model that could be needed (including models that may be used very seldomly). Additionally, making modifications to these models is more complex and introduces additional overhead in deploying these changes to avoid them from impacting existing **ClinicalModelJobs**.

Per-model workers improve specialization but undermine M2’s rapid run-time evolution: every new/updated model becomes a deployment and operations task. This fragments capacity under P2 (capacity must be managed per worker type), increasing queueing risk for rare models and complicating overload handling. The database-driven, runtime-loaded approach avoids that operational coupling while preserving predictable scheduling behavior.

Architectural Decision 2: Local caching with explicit invalidation

To ensure that updates to clinical models do not interfere with ongoing risk calculations, clinical models and their configurations are cached locally in the `ClinicalModelCache`. This guarantees that running risk estimation jobs continue to use the same model version throughout their execution, even when a newer version is deployed.

Because assignments of clinical models and configurations are expected to change infrequently, local caching significantly reduces database access and improves evaluation performance. Cache consistency is maintained through explicit invalidation, triggered by the `OtherFunctionality` component, rather than through time-based expiration.

Changes to clinical models affect only the risk estimation subsystem, including the model cache, scheduler, processors, and combiner components. All other subsystems remain unaffected. Deployment of new or modified

clinical models is lightweight and completes within 30 minutes, as it involves only database updates followed by cache invalidation.

To support consistent aggregation of risk estimation results, all clinical models are required to produce outputs on a common risk scale. This enables the `RiskEstimationCombiner` to combine results deterministically using predefined model weights.

Sensitivity points

Caching avoids repeated remote database access and ensures that previous versions remain available for ongoing computations. This introduces a *sensitivity point* with respect to timing and coordination: system behavior becomes sensitive to when invalidation is triggered and how quickly caches converge, especially under overload conditions.

Relation to other architectural decisions

Caching reinforces P2: Risk estimation by clinical models by reducing per-job latency and protecting the scheduler and processors from excessive database access. At the same time, explicit invalidation introduces additional control traffic that may coincide with peak load conditions managed by the scheduler. This creates a direct trade-off between throughput predictability (P2) and deployment flexibility (M2).

Trade-off justification

The architecture accepts this trade-off by defining a clear stability boundary: running jobs operate on a snapshot of cached models, while updates only affect jobs started after invalidation. The architecture prioritizes clinical safety and continuity over strict instantaneous global uniformity.

Considered alternatives

Custom combination logic for combining clinical model risk results. The suggested solution imposes an important constraint on the result of the `ClinicalModelJobs` by requiring risk results on the same common scale. While providing each `ClinicalModel` the ability to express the results in its own risk unit increases the flexibility of the analyses, it would introduce considerable complexity in the combination of the results of the clinical models, especially as new clinical models are introduced that would lead to new unforeseen combinations that need to be calculated.

Allowing model-specific risk units increases flexibility, but it forces new conversion/combination rules whenever models are added or updated, which scales poorly with M2. This also weakens P2's predictability and clinical explainability because the combined score becomes harder to validate and reproduce across evolving model sets. A common risk scale trades expressiveness for deterministic, testable aggregation.

Supporting in-place modification of clinical models. The in-place modification of `ClinicalModels` would be desirable for introducing minor bug fixes without having to create separate new versions of clinical models. However, to guarantee that changes to clinical models do not have any impact on existing `ClinicalModelJobs`, changes to existing clinical models are not allowed. Therefore, even minor changes will require the introduction of new version of the clinical model. This ensures that existing jobs are unaffected, the results are predictable and reproducible.

In-place modification conflicts with M2's requirement that ongoing calculations are unaffected: with P2 concurrency, in-flight jobs could observe changed logic/configuration. Even with transactional updates, auditability and reproducibility degrade because the exact model version used per job becomes harder to reconstruct. Append-only versioning preserves stable execution snapshots and clinically defensible traceability.

A. Client-Server View

Figures

A.1 Context diagram for the client-server view	10
A.2 Primary diagram of the client-server view	10

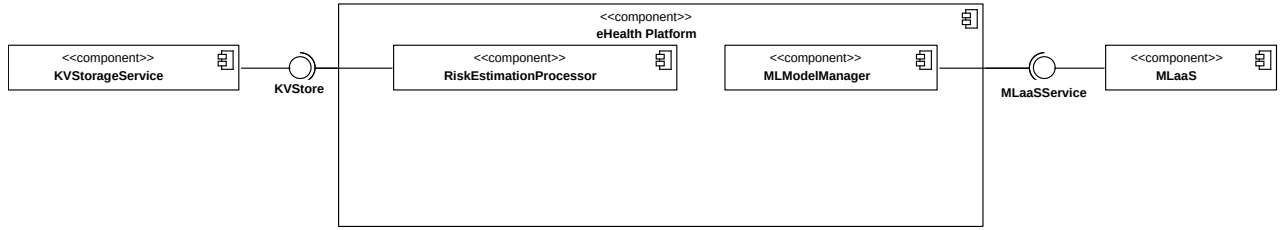


Figure A.1: Context diagram for the client-server view

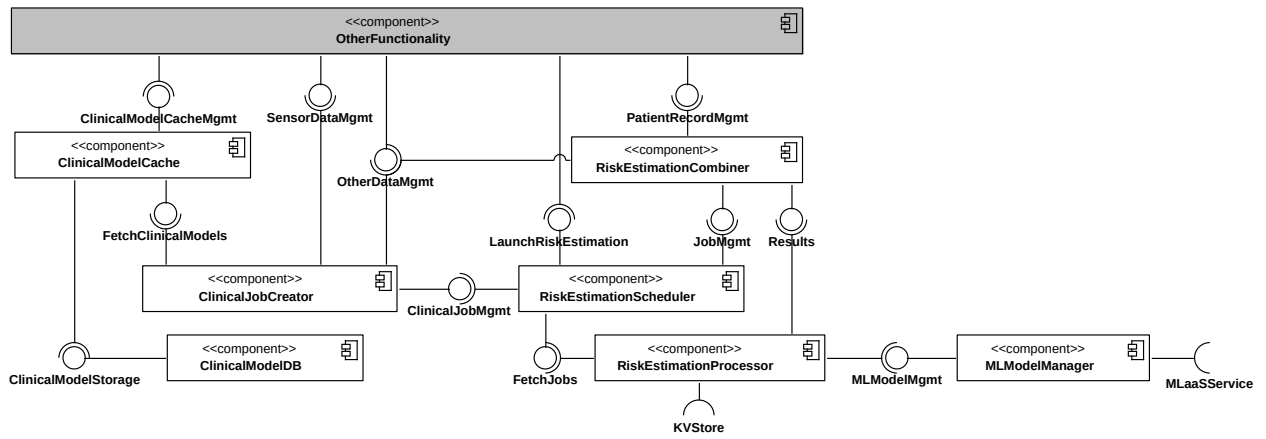


Figure A.2: Primary diagram of the client-server view

This architecture is not yet complete. Functionality that still has to be worked out in more detail is captured by the OtherFunctionality component in gray.

B. Decomposition View

Figures

B.1	Context diagram for the decomposition view	11
B.2	Decomposition of the MLModelManager	11
B.3	Decomposition of the RiskEstimationProcessor	12

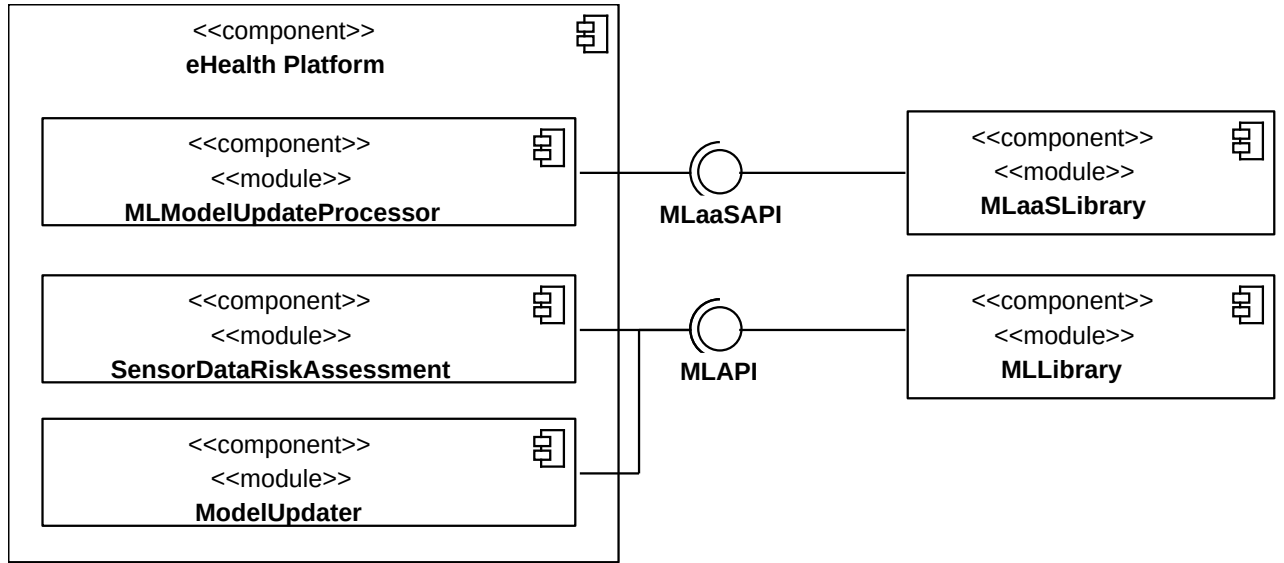


Figure B.1: Context diagram for the decomposition view

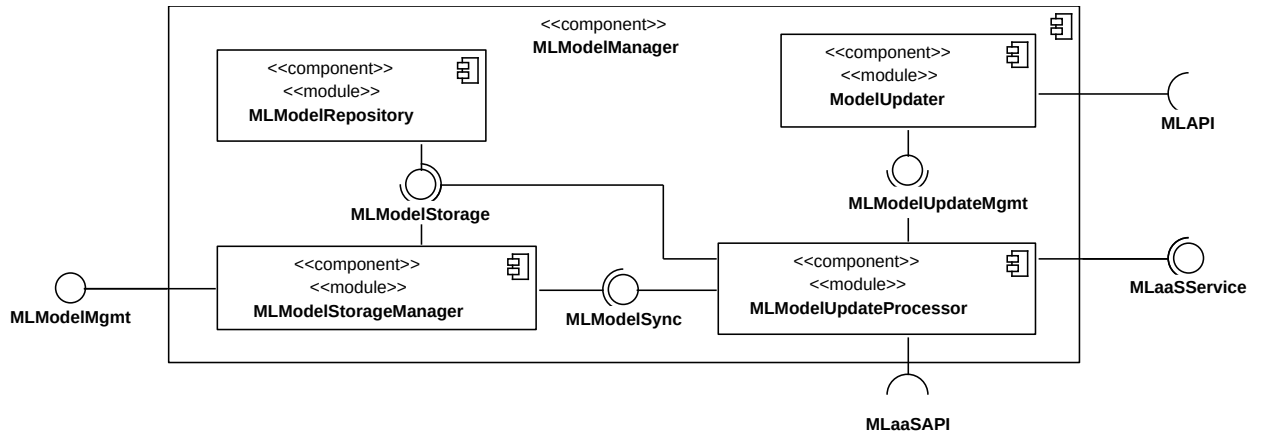


Figure B.2: Decomposition of the MLModelManager

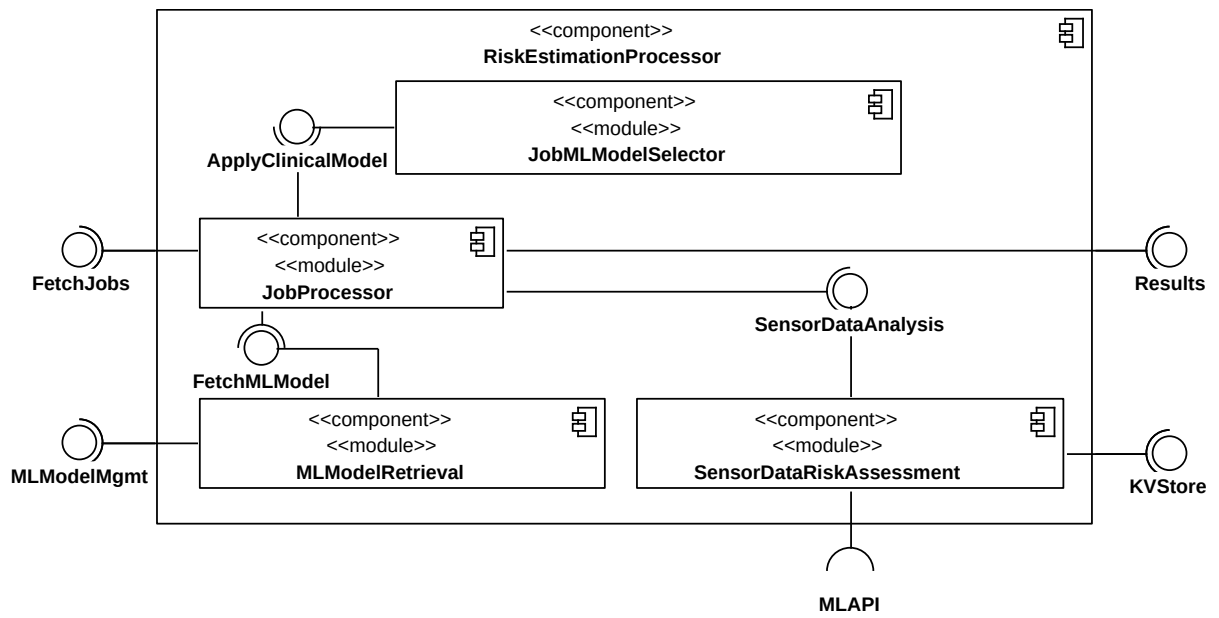


Figure B.3: Decomposition of the RiskEstimationProcessor

C. Deployment view

Figures

C.1	Context diagram for the deployment view	13
C.2	Primary deployment diagram	13
C.3	Pilot Deployment (200 patients, 5 clin models)	14
C.4	Development Test Deployment (20 patients, 3 clin models)	14

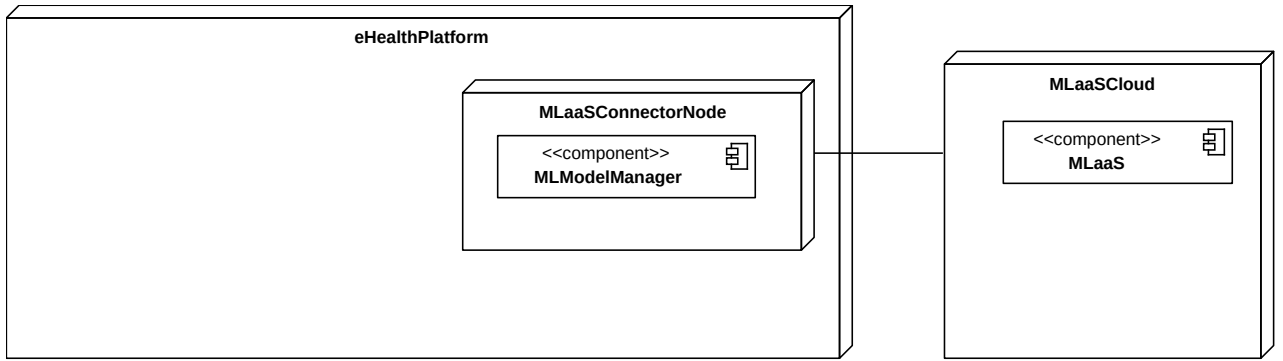


Figure C.1: Context diagram for the deployment view

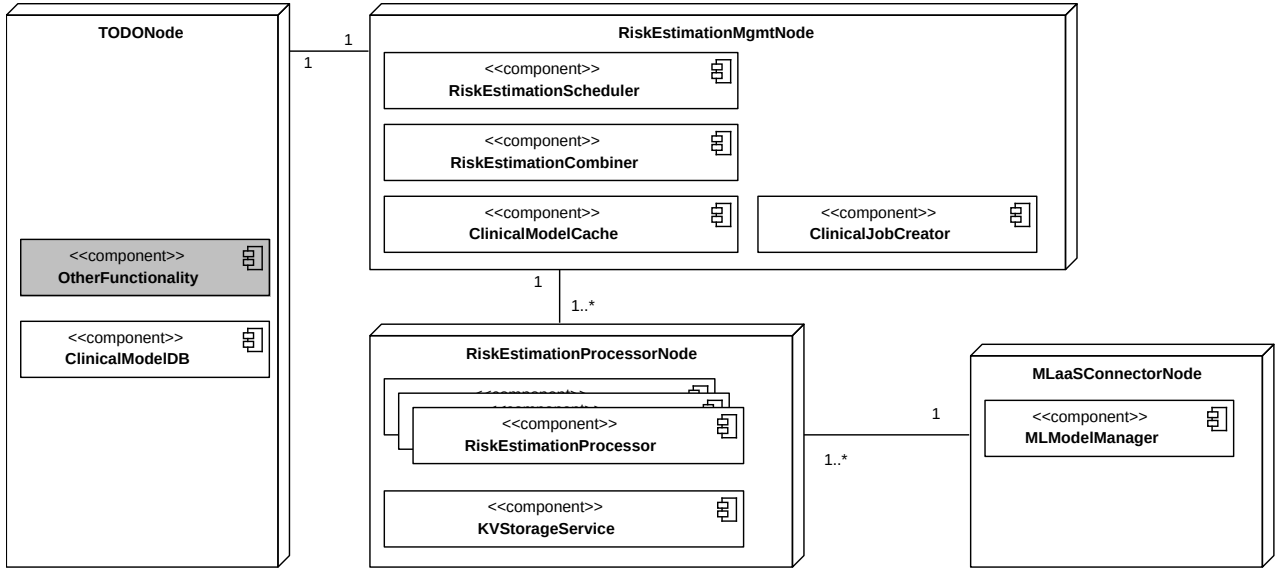


Figure C.2: Primary deployment diagram

This deployment is necessarily incomplete as the architecture is incomplete. The functionality that is not worked out yet is captured by **OtherFunctionality** component, currently deployed on **TODONode**. This **TODONode** will also have to be replaced with the final nodes are required by the replacement of **OtherFunctionality**.

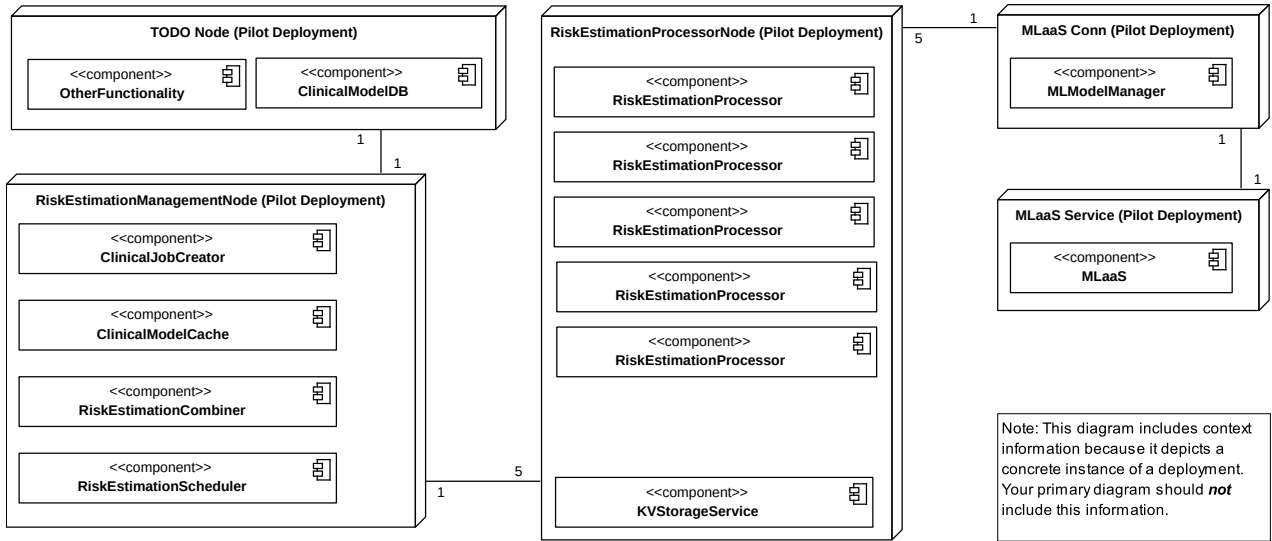


Figure C.3: Pilot Deployment (200 patients, 5 clin models)

This diagram shows a concrete initial pilot deployment for 200 patients with an average of 5 clinical models per patient. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

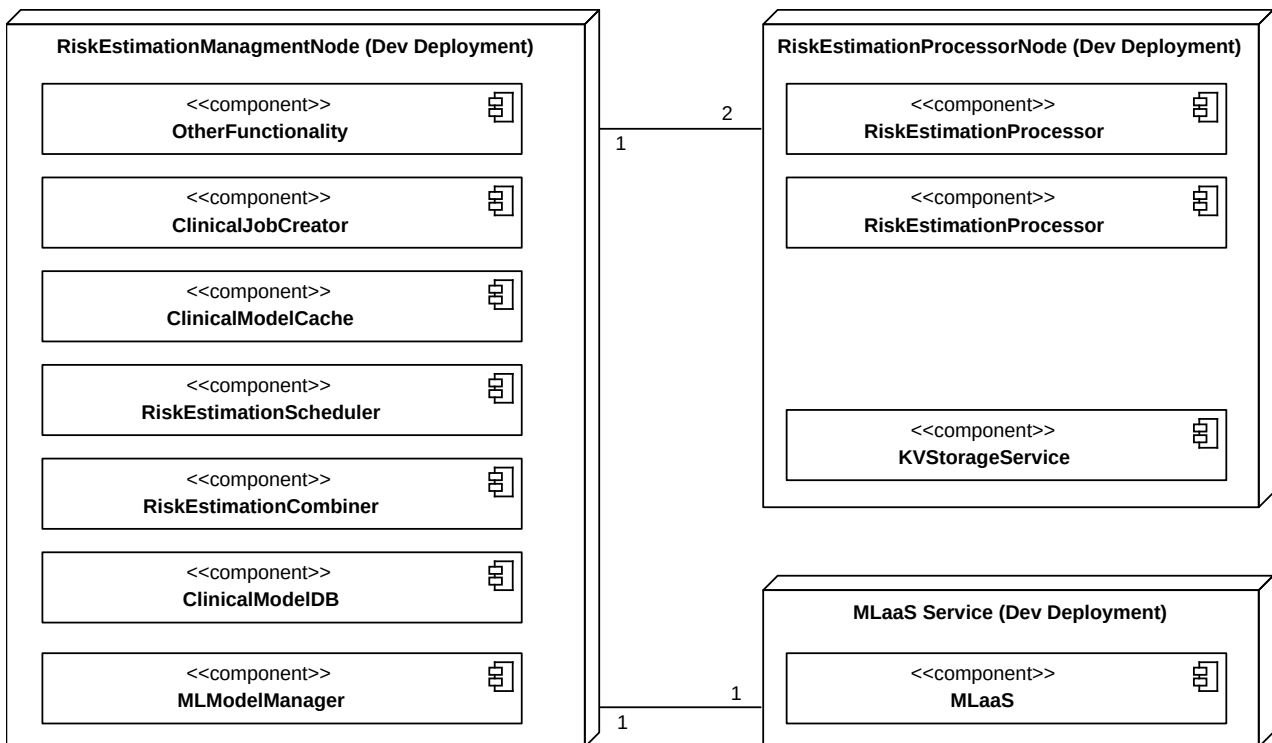


Figure C.4: Development Test Deployment (20 patients, 3 clin models)

This diagram shows a simplified deployment for local development. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

D. Process View

Figures

D.1	Risk estimation process	15
D.2	Compute clinical model result	16
D.3	MLModel Synchronization	16
D.4	Processing MLModel updates	17
D.5	Incoming sensor data	17
D.6	Combining ClinicalModelJob results	18

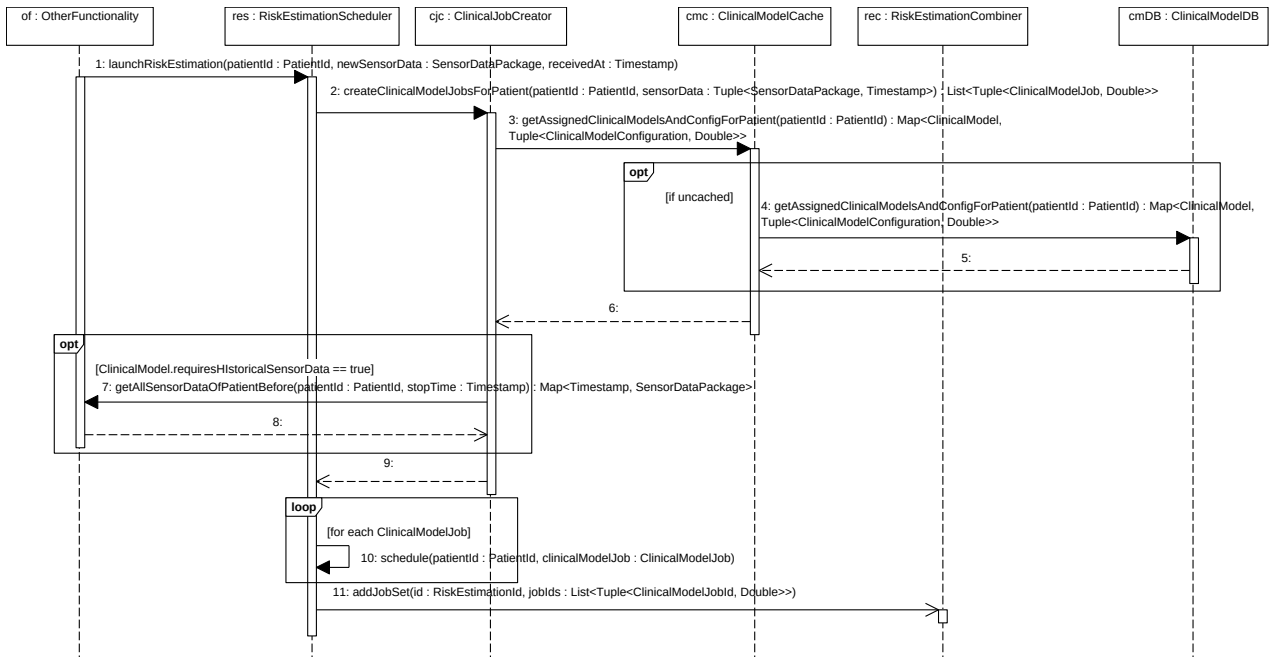


Figure D.1: Risk estimation process

The scheduling and execution of a risk estimation triggered by the arrival of new sensor data. Note that if one or more clinical models in the **ClinicalModelCache** have been invalidated, all relevant clinical models are renewed.

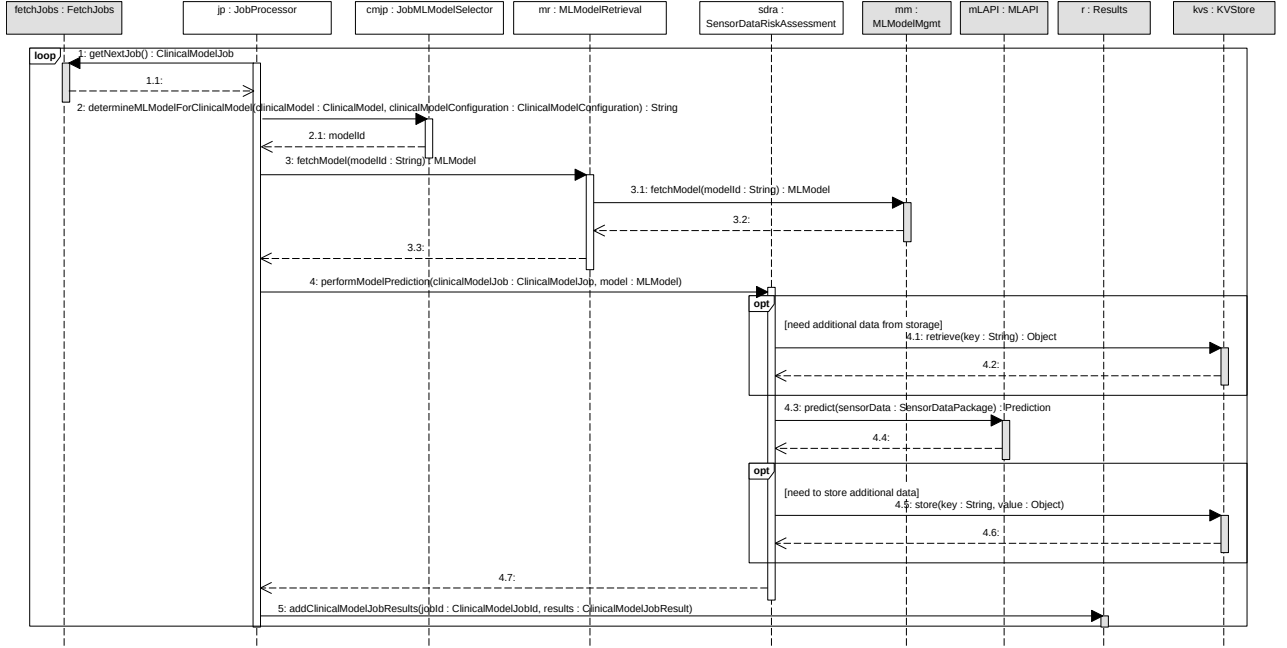


Figure D.2: Compute clinical model result

This sequence diagram shows the detailed application and computation of the clinical model. The first step involves the retrieval of the appropriate **MLModel** (specified by the **ClinicalModel**). The second step involves the calculation of the prediction using the retrieved **MLModels**. Note that, as this diagram depicts the internal flow between modules within the **RiskEstimationProcessor**, the gray lifelines depict its (external) interfaces.

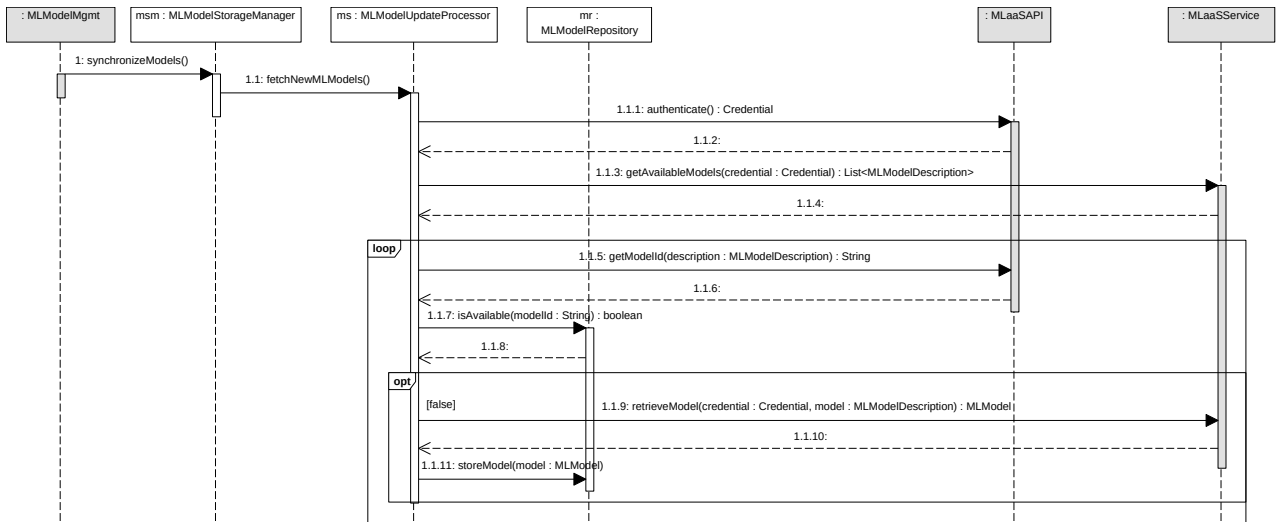


Figure D.3: MLModel Synchronization

As the system operations new **MLModels** may become available for use by the **ClinicalModels**. This diagram illustrates the synchronization logic to fetch any unavailable **MLModels** and to store a local copy of the **MLModels** in the **MLModelRepository** for faster retrieval and use in the sensor data risk assessment. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

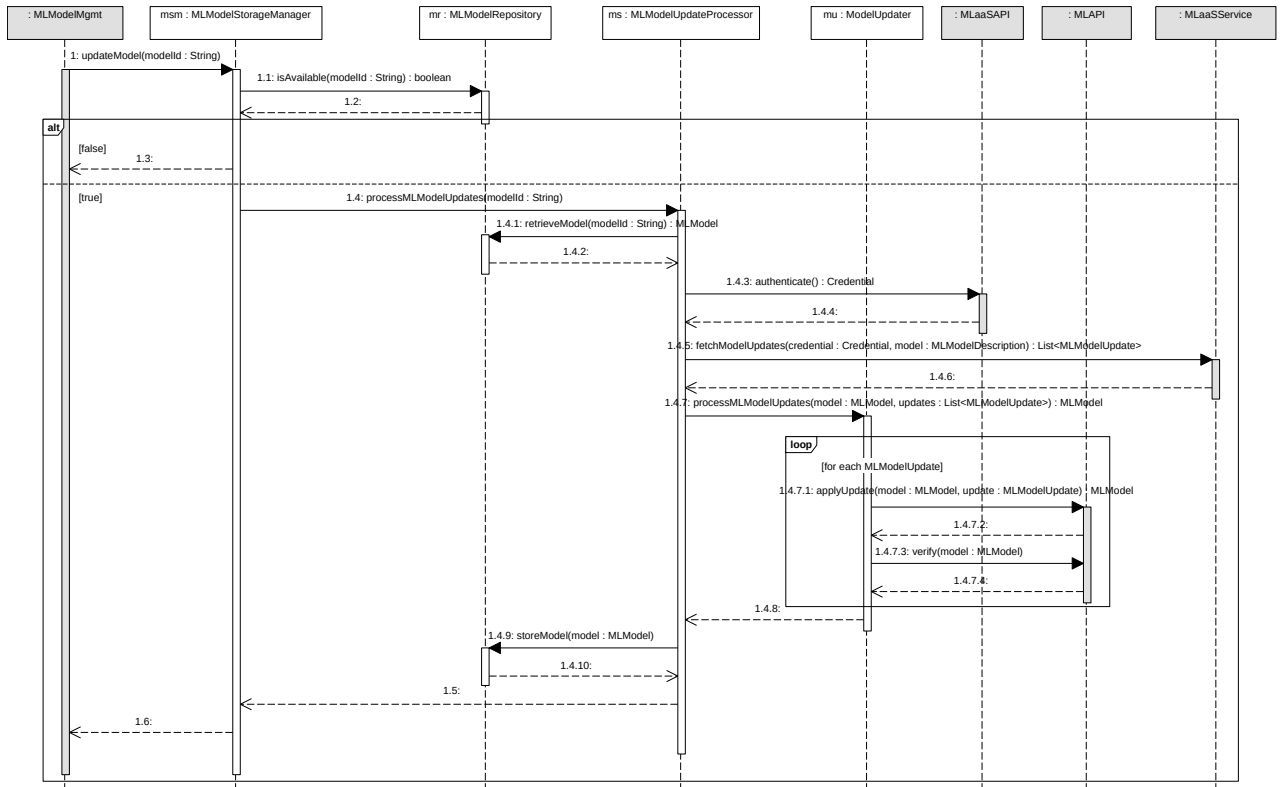


Figure D.4: Processing MLModel updates

MLModelUpdates are made available to improve the existing **MLModels**. This diagram illustrates the retrieval of these **MLModelUpdates** and their application on the local **MLModel**. As updates to these models influence the risk assessments, they need to be triggered for each individual **MLModel** that requires an update. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

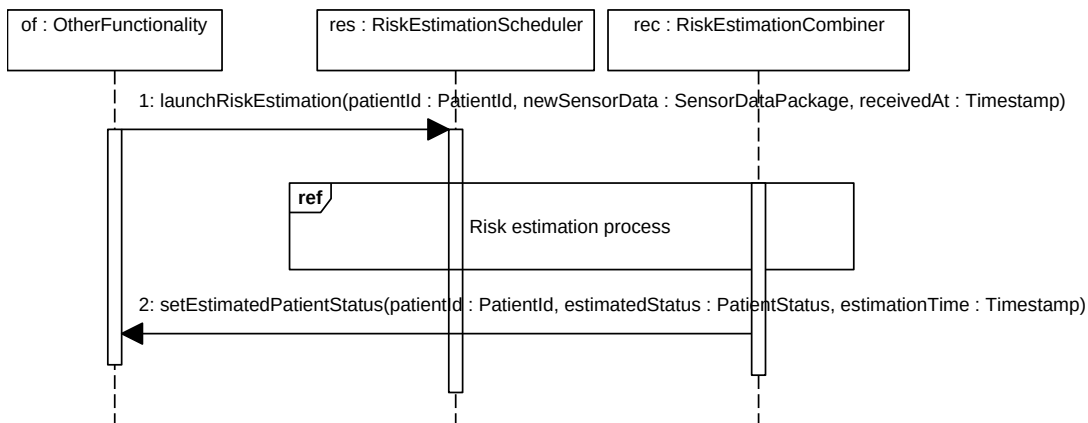


Figure D.5: Incoming sensor data

This diagram shows a small part of the processing of incoming sensor data. It does not yet depict the arrival of the sensor data, storage in a database, or any other processing, as this functionality is not yet worked out.

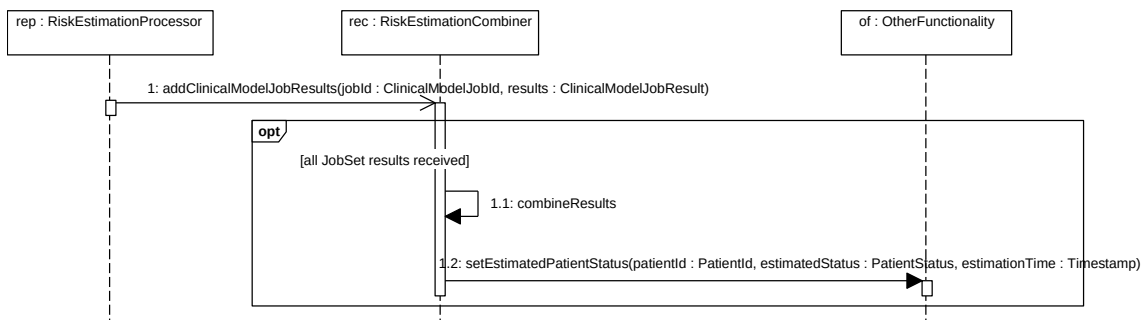


Figure D.6: Combining ClinicalModelJob results

E. Element catalog

E.1 Components

E.1.1 ClinicalJobCreator

Responsibility: The `ClinicalJobCreator` is responsible for constructing the **ClinicalModelJob** objects and populating them with the necessary patient health data for execution by the `RiskEstimationProcessors`.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces:  `ClinicalJobMgmt`

Required interfaces:  `FetchClinicalModels`,  `OtherDataMgmt`,  `SensorDataMgmt`

Deployed on:  `RiskEstimationManagementNode` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `RiskEstimationMgmtNode`

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.2 ClinicalModelCache

Responsibility: The `ClinicalModelCache` is a read-through cache responsible for caching the **ClinicalModels** that should be evaluated for a certain patient and their configurations. This cache is located close to the `RiskEstimationProcessor` and `RiskEstimationCombiner` in order to improve the latency of the whole risk estimation flow. The items in the `ClinicalModelCache` do not expire over time, but should be invalidated explicitly if needed. Cached models remain available for the `RiskEstimationProcessor` while updates are made in the `ClinicalModelDB`.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces:  `ClinicalModelCacheMgmt`,  `FetchClinicalModels`

Required interfaces:  `ClinicalModelStorage`,  `OtherDataMgmt`

Deployed on:  `RiskEstimationManagementNode` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `RiskEstimationMgmtNode`

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.3 ClinicalModelDB

Responsibility: The `ClinicalModelDB` stores all the **ClinicalModels** and the configurations of these **ClinicalModels** for the different patients separately from other data. It only allows appending new **ClinicalModels**.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces:  `ClinicalModelStorage`

Required interfaces: None

Deployed on:  `TODO Node` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `TODO Node`

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.4 eHealth Platform

Responsibility: The parent `eHealth Platform` component that represents the context boundary in the client-server view.

Super-components: None

Sub-components:  `OtherFunctionality`,  `MLModelManager`,  `ClinicalModelCache`,  `RiskEstimationCombiner`,  `RiskEstimationProcessor`,  `RiskEstimationScheduler`,  `ClinicalJobCreator`,  `ClinicalModelDB`

Provided interfaces: None

Required interfaces: $\prec$ KVStore, $\prec$ MLaaSService

Deployed on: ☐ TODO Node (Pilot Deployment), ☐ RiskEstimationManagementNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ MLaaS Conn (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ MLaaSConnectorNode, ☐ RiskEstimationMgmtNode, ☐ RiskEstimationProcessorNode, ☐ TODONode

Visible on diagrams: figs. A.1 and B.1

E.1.5 HIS

Responsibility: Hospital Information System

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ healthAPI

Required interfaces: None

Deployed on: Nowhere

Visible on diagrams: None

E.1.6 KVStorageService

Responsibility: Service for key-value storage of data.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ KVStore

Required interfaces: None

Deployed on: ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ RiskEstimationProcessorNode

Visible on diagrams: figs. A.1, C.2, C.3 and C.4

E.1.7 MLaaS

Responsibility: MachineLearning-as-a-Service cloud provider. This cloud service provides APIs for managing different **MLModels** and exchanging **MLModel** updates with other parties.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ MLaaSService

Required interfaces: None

Deployed on: ☐ MLaaS Cloud, ☐ MLaaS Service (Pilot Deployment), ☐ MLaaS Service (Dev Deployment)

Visible on diagrams: figs. A.1, C.1, C.3 and C.4

E.1.8 MLModelManager

Responsibility: This component is responsible for managing the local **MLModels** for making predictions for malignant events based on the incoming sensor data. It keeps track of the available **MLModels** that can be used and supports exchanging **MLModel** updates with other systems using the APIs of MLaaS providers.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ MLModelMgmt

Required interfaces: $\prec$ MLaaSService

Deployed on: ☐ MLaaS Conn (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ MLaaSConnectorNode

Visible on diagrams: figs. A.1, A.2, B.2, C.1, C.2, C.3 and C.4

E.1.9 OtherFunctionality

Responsibility: Other functionality that still has to be worked out.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ OtherDataMgmt, $\rightarrow$ PatientRecordMgmt, $\rightarrow$ SensorDataMgmt

Required interfaces: $\leftarrow$ ClinicalModelCacheMgmt, $\leftarrow$ LaunchRiskEstimation

Deployed on: ☐ TODO Node (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ TODO Node

Visible on diagrams: figs. A.2, C.2, C.3, C.4, D.1, D.5 and D.6

E.1.10 RiskEstimationCombiner

Responsibility: The RiskEstimationCombiner is responsible for combining the results of the **ClinicalModelJobs** belonging to a patient risk estimation. More precisely, the RiskEstimationScheduler passes the set of the scheduled jobs for a risk estimation to the RiskEstimationCombiner before scheduling them. The RiskEstimationCombiner then waits for all results to arrive, combines them, and propagates the new sensor data and the results of the risk estimation to the patient record if needed.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ JobMgmt, $\rightarrow$ Results

Required interfaces: $\leftarrow$ OtherDataMgmt, $\leftarrow$ PatientRecordMgmt

Deployed on: ☐ RiskEstimationManagementNode (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.2, C.3, C.4, D.1, D.5 and D.6

E.1.11 RiskEstimationProcessor

Responsibility: The RiskEstimationProcessor is responsible for computing **ClinicalModelJobs**. The RiskEstimationProcessor fetches new **ClinicalModelJobs** from the RiskEstimationScheduler, uses the specified **MLModel** to calculate the risk prediction, and passes the result to the RiskEstimationCombiner. Multiple instances of the RiskEstimationProcessor run in parallel to improve the throughput.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: None

Required interfaces: $\leftarrow$ FetchJobs, $\leftarrow$ KVStore, $\leftarrow$ MLModelMgmt, $\leftarrow$ Results

Deployed on: ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ RiskEstimationProcessorNode

Visible on diagrams: figs. A.1, A.2, B.3, C.2, C.3, C.4 and D.6

E.1.12 RiskEstimationScheduler

Responsibility: The RiskEstimationScheduler is responsible for taking in new and scheduling requests for risk estimations. It keeps track of the throughput of incoming jobs and changes the scheduling from first-in/first-out to dynamic priority earliest deadline first when going into overload modus.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ FetchJobs, $\rightarrow$ LaunchRiskEstimation

Required interfaces: $\leftarrow$ ClinicalJobMgmt, $\leftarrow$ FetchClinicalModels, $\leftarrow$ JobMgmt, $\leftarrow$ OtherDataMgmt, $\leftarrow$ SensorDataMgmt

Deployed on: ☐ RiskEstimationManagementNode (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.2, C.3, C.4, D.1 and D.5

E.2 Modules

E.2.1 JobMLModelSelector

Responsibility: This module is responsible for determining the appropriate **MLModel** to use for making the prediction on the provided sensor data, as there could be multiple **MLModel** candidates for

making predictions.

Super-components:  RiskEstimationProcessor  eHealth Platform

Super-modules: None

Sub-modules: None

Provided interfaces:  ApplyClinicalModel

Required interfaces: None

Visible on diagrams: figs. B.3 and D.2

E.2.2 JobProcessor

Responsibility: The JobProcessor module coordinates the execution of the **ClinicalModelJobs** to ensure that the required **MLModels** are fetched, and the prediction is performed with the appropriate **MLModel**.

Super-components:  RiskEstimationProcessor  eHealth Platform

Super-modules: None

Sub-modules: None

Provided interfaces: None

Required interfaces:  ApplyClinicalModel,  FetchJobs,  FetchMLModel,  Results,  SensorData-Analysis

Visible on diagrams: figs. B.3 and D.2

E.2.3 MLaaSLibrary

Responsibility: MLaaS development library for working with the MLaaS providers. It supports storage & retrieval of **MLModels**, and exchanging **MLModelUpdates** with MLaaS providers.

Super-components: None

Super-modules: None

Sub-modules: None

Provided interfaces:  MLaaSAPI

Required interfaces: None

Visible on diagrams: fig. B.1

E.2.4 MLLibrary

Responsibility: Library for working with **MLModels** locally and using them to make predictions based on the incoming sensor data. The library can also process external updates to apply them to a specific **MLModel**.

Super-components: None

Super-modules: None

Sub-modules: None

Provided interfaces:  MLAPI

Required interfaces: None

Visible on diagrams: fig. B.1

E.2.5 MLModelRepository

Responsibility: This module stores **MLModels** locally and makes them available for use by the RiskEstimationProcessor when processing **ClinicalModelJobs**.

Super-components:  eHealth Platform  MLModelManager

Super-modules: None

Sub-modules: None

Provided interfaces:  MLModelStorage

Required interfaces: None

Visible on diagrams: figs. B.2, D.3 and D.4

E.2.6 MLModelRetrieval

Responsibility: This module is responsible for retrieving the requested **MLModels** from the MLModelManager.

Super-components: ¶ RiskEstimationProcessor ▷ ¶ eHealth Platform
Super-modules: None
Sub-modules: None
Provided interfaces: ↗ FetchMLModel
Required interfaces: ↖ MLModelMgmt
Visible on diagrams: figs. B.3 and D.2

E.2.7 MLModelStorageManager

Responsibility: This module is responsible for managing the locally stored **MLModels**, triggering the synchronization and retrieval of new **MLModels**, and coordinating the updates to the existing **MLModels**.

Super-components: ¶ eHealth Platform ▷ ¶ MLModelManager
Super-modules: None
Sub-modules: None
Provided interfaces: ↗ MLModelMgmt
Required interfaces: ↖ MLModelStorage, ↖ MLModelSync
Visible on diagrams: figs. B.2, D.3 and D.4

E.2.8 MLModelUpdateProcessor

Responsibility: The MLModelUpdater processes and applies updates to the local **MLModels** to prevent updates to these models from requiring to retrieve the full **MLModels**.

Super-components: ¶ eHealth Platform ▷ ¶ MLModelManager
Super-modules: None
Sub-modules: None
Provided interfaces: ↗ MLModelSync
Required interfaces: ↖ MLaaSAPI, ↖ MLaaSService, ↖ MLModelStorage, ↖ MLModelUpdateMgmt
Visible on diagrams: figs. B.1, B.2, D.3 and D.4

E.2.9 ModelUpdater

Responsibility: This module is responsible for processing **MLModelUpdates** and applying them to the **MLModel**.

Super-components: ¶ eHealth Platform ▷ ¶ MLModelManager
Super-modules: None
Sub-modules: None
Provided interfaces: ↗ MLModelUpdateMgmt
Required interfaces: ↖ MLAPI
Visible on diagrams: figs. B.1, B.2 and D.4

E.2.10 SensorDataRiskAssessment

Responsibility: This module performs the actual risk assessment by using the **MLModel** to make a prediction on the sensor data.

Super-components: ¶ RiskEstimationProcessor ▷ ¶ eHealth Platform
Super-modules: None
Sub-modules: None
Provided interfaces: ↗ SensorDataAnalysis
Required interfaces: ↖ KVStore, ↖ MLAPI
Visible on diagrams: figs. B.1, B.3 and D.2

E.3 Interfaces

E.3.1 ApplyClinicalModel

Provided by: ¶ JobMLModelSelector
Required by: ¶ JobProcessor

Operations:

- String `determineMLModelForClinicalModel(ClinicalModel clinicalModel, ClinicalModelConfiguration clinicalModelConfiguration)`
 - Effect: Apply the **ClinicalModel** to obtain the most appropriate **MLModel** to use for the calculation.
 - Returns: **Identifier** of the **MLModel** to use.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.2 ClinicalJobMgmt

Provided by:  ClinicalJobCreator

Required by:  RiskEstimationScheduler

Operations:

- List<Tuple<**ClinicalModelJob**, Double>> `createClinicalModelJobsForPatient(PatientId patientId, Tuple<SensorDataPackage, Timestamp> sensorData)`
 - Effect: Construct the list of **ClinicalModelJobs** for a specified patient. These jobs are populated with the necessary data for their execution by the **RiskEstimationProcessor**.
 - Returns: A list of tuples containing the **ClinicalModelJob** and its weight.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.3 ClinicalModelCacheMgmt

Provided by:  ClinicalModelCache

Required by:  OtherFunctionality

Operations:

- void `invalidateCacheEntries(PatientId patientId)`
 - Effect: The **ClinicalModelCache** will invalidate (i.e., remove) all items in its cache for the patient identified by `patientId`. If the cache does not contain any items for this patient, nothing is changed. After invalidating the cached items for a certain patient, the next request for them will lead to fetching them from the database and storing them in the cache again.
 - Sequence Diagrams: None

Diagrams: None

E.3.4 ClinicalModelStorage

Provided by:  ClinicalModelDB

Required by:  ClinicalModelCache

Operations:

- Map<**ClinicalModel**, Tuple<**ClinicalModelConfiguration**, Double>> `getAssignedClinicalModelsAndConfigForPatient(PatientId patientId)` throws *NoSuchPatientException*
 - Effect: Fetch and return the ids of the **ClinicalModels** assigned to the patient identified by `patientId` and their configurations for this patient. The configurations are returned as a **ClinicalModelConfiguration** containing a map of configuration parameters and their value.
 - Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.5 FetchClinicalModels

Provided by:  ClinicalModelCache

Required by:  ClinicalJobCreator,  RiskEstimationScheduler

Operations:

- Map<**ClinicalModel**, Tuple<**ClinicalModelConfiguration**, Double>> `getAssignedClinicalModelsAndConfigForPatient(PatientId patientId)`

- Effect: The **ClinicalModelCache** will return the clinical models assigned to the patient identified by `patientId` and their configurations for this patient. The configurations are a map of configuration parameters and their value. If the cache contains data for the patient with given id, the data from the cache is returned. If not, the **ClinicalModelCache** will fetch all clinical models assigned to the patient with given `patientId` and their configurations for this patient, store these in the cache and return them.
- Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
- Sequence Diagrams: fig. D.1

Diagrams: None

E.3.6 FetchJobs

Provided by: `⌘ RiskEstimationScheduler`

Required by: `⌘ JobProcessor`, `⌘ RiskEstimationProcessor`

Operations:

- **ClinicalModelJob** `getNextJob()`
 - Effect: Returns the next clinical model computation job that must be performed (i.e., the first job in the queue).
 - Returns: The next clinical model computation job that must be performed.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.7 FetchMLModel

Provided by: `⌘ MLModelRetrieval`

Required by: `⌘ JobProcessor`

Operations:

- **MLModel** `fetchModel(String modelId)` throws *NoSuchMLModelException*
 - Effect: Fetch the **MLModel** corresponding with the provided `modelId`.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.8 healthAPI

Provided by: `⌘ HIS`

Required by: None

Description : This interface provides a reduced set of simplified operations that are sufficient in the context of the patient monitoring service that is being developed. It is derived from the HL7 FHIR standard for healthcare interoperability, based on the HAPI FHIR implementation of the FHIR specification in Java. The relevant OpenAPIs are described at: <https://hapi.fhir.org/baseR4/swagger-ui/>, while the FHIR standard is available at: <https://hl7.org/fhir/>. The descriptions from the data types used in this interface follow directly from the API documentation (<https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r5/org/hl7/fhir/r5/model/package-summary.html>).

Operations:

- `void deleteObservation(string id)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete an **Observation** instance with the specified id.
 - Sequence Diagrams: None
- `void deletePatient(string id)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete a **Patient** instance with the specified id.
 - Sequence Diagrams: None
- `void deleteRiskAssessment(string id)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete a **RiskAssessment** instance with the specified id.
 - Sequence Diagrams: None
- **Observation** `getObservation(string id)` throws *AuthorizationException2*, *AuthenticationException2*

- Effect: Retrieve an observation instance
- Returns: The **Observation** with the specified identifier.
- Sequence Diagrams: None
- **Patient** `getPatient(string id)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Retrieve a patient record with the specified identifier.
 - Returns: The **Patient** with the specified identifier.
 - Sequence Diagrams: None
- **RiskAssessment** `getRiskAssessment(string id)` throws *AuthorizationException2*
 - Effect: Retrieve a **RiskAssessment** instance with the specified id.
 - Returns: The **RiskAssessment** with the specified identifier.
 - Sequence Diagrams: None
- **Observation** `saveObservation(Observation observation)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Create a new **Observation** instance or in case an existing **Observation** instance exists with the same identifier, update this **Observation** instance.
 - Returns: The **Observation** object.
 - Sequence Diagrams: None
- **Patient** `savePatient(Patient patient)` throws *AuthorizationException2*
 - Effect: Create a new **Patient** instance or update an existing **Patient** instance.
 - Returns: The updated **Patient** object.
 - Sequence Diagrams: None
- **RiskAssessment** `saveRiskAssessment(RiskAssessment riskAssessment)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Create a new **RiskAssessment** instance or update an existing **RiskAssessment** instance.
 - Returns: The updated **RiskAssessment** object.
 - Sequence Diagrams: None
- `List<Observation> searchObservation(Query<Date> date, Query<String> dataAbsentReason, Query<Patient> subject, Query<String> valueConcept, Query<Date> valueDate, Query<Observation> derivedFrom, Query<Patient> patient, Query<Quantity> valueQuantity, Query<String> identifier, Query<String> performer, Query<String> method, Query<String> category, Query<String> device, Query<ObservationStatus> status)` throws *AuthorizationException2*
 - Effect: Search for **Observation** instances matching the specified query criteria.
 - Returns: The list of **Observations** matching the specified search criteria.
 - Sequence Diagrams: None
- `List<Patient> searchPatient(Query<Date> birthDate, Query<Boolean> deceased, Query<String> addressState, Query<String> administrativeGender, Query<String> link, Query<String> language, Query<String> addressCountry, Query<Date> deathDate, Query<String> phonetic, Query<String> telecom, Query<String> addressCity, Query<String> email, Query<String> given, Query<String> identifier, Query<String> address, Query<String> generalPractitioner, Query<Boolean> active, Query<String> addressPostalCode, Query<String> phone, Query<String> organizationCustodian, Query<String> name, Query<String> family)` throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Search for **Patient** instances matching the specified query criteria.
 - Returns: The list of **Patients** matching the specified search criteria.
 - Sequence Diagrams: None
- `List<RiskAssessment> searchRiskAssessment(Query<Date> date, Query<String> identifier, Query<String> performer, Query<String> method, Query<Range> probability, Query<Patient> subject, Query<String> condition, Query<Patient> patient, Query<BigDecimal> risk)` throws *AuthenticationException2*, *AuthorizationException2*
 - Effect: Search for a **RiskAssessment** instance matching the provided query criteria.
 - Returns: The list of **RiskAssessments** matching the specified search criteria.
 - Sequence Diagrams: None

Diagrams: None

E.3.9 JobMgmt

Provided by:  RiskEstimationCombiner

Required by:  RiskEstimationScheduler

Operations:

- `void addJobSet(RiskEstimationId id, List<Tuple<ClinicalModelJobId, Double>> jobIds)`
 - Effect: Adds a set of identifiers for jobs belonging to a single risk estimation identified by id. The partial results of each clinical model computation job identified by an element in jobIds have to be combined in order to find the final result of the risk estimation as a whole.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.10 KVStore

Provided by: `KVStorageService`

Required by: `RiskEstimationProcessor`, `SensorDataRiskAssessment`, `eHealth Platform`

Operations:

- `Object retrieve(String key)` throws *IOException*
 - Effect: Retrieve the object with the provided key from the Key-Value storage.
 - Returns: The object retrieved from storage.
 - Sequence Diagrams: fig. D.2
- `void store(String key, Object value)` throws *IOException*
 - Effect: Store the provided object in the Key-Value storage under the provided key.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.11 LaunchRiskEstimation

Provided by: `RiskEstimationScheduler`

Required by: `OtherFunctionality`

Operations:

- `void launchRiskEstimation(PatientId patientId, SensorDataPackage newSensorData, Timestamp receivedAt)`
 - Effect: The `RiskEstimationScheduler` will fetch the clinical models and their configurations associated to the patient identified by patientId from storage using the `ClinicalModelCache`, notify the `RiskEstimationCombiner` of the different jobs that will be performed for a single risk estimation and schedule the individual jobs in its queue. In normal modus, queued jobs are returned in FIFO order. In overload modus, the system switches to dynamic priority: earliest deadline first and enqueues jobs of patient with a high risk level with an earlier deadline (2 instead of 5 minutes) to prioritize them over patients with lower risk levels. The `SensorDataPackage` newSensorData is passed because its arrival triggered the risk estimation. A risk level is estimated based on the computation of these clinical models. For the computation of the clinical models, the given sensor data newSensorData (which is the new sensor data that was received) is used and other required data is fetched from the respective databases if needed. The given time-stamp receivedAt is used in order to avoid fetching the new sensor data from the database. This time-stamp represents the time at which the new sensor data was received.
 - Sequence Diagrams: figs. D.1 and D.5

Diagrams: None

E.3.12 MLaaSAPI

Provided by: `MLaaSLibrary`

Required by: `MLModelUpdateProcessor`

Description : This API provides a number of operations for managing the models deployed on the MLaaS cloud.

Operations:

- `Credential authenticate()` throws *InvalidAPIKeyException*
 - Effect: Construct a **Credential** object for authenticating requests with the MLaaS service provider. This operation uses the previously configured API key to construct the **Credential** object. If such an API key is not configured, an exception will be thrown.
 - Returns: Authentication token for subsequent requests.
 - Sequence Diagrams: figs. D.3 and D.4
- `void delete(MLModel model, Credential token)` throws *AuthorizationException*

- Effect: Delete a model from the online MLaaS service.
- Sequence Diagrams: None
- **MLModelDescription** getModelDescription(List<**MLModelDescription**> descriptions, String modelId)
 - Effect: This method retrieves the correct **MLModelDescription** that corresponds with the **MLModel** identified by the provided modelId.
 - Returns: The **MLModelDescription** corresponding with the **MLModel** identified by the modelId.
 - Sequence Diagrams: None
- String getModelId(**MLModelDescription** description)
 - Effect: Extract the modelId from the **MLModelDescription**.
 - Returns: String with the modelId of the **MLModel** specified in the **MLModelDescription**.
 - Sequence Diagrams: fig. D.3
- void register(**MLModel** model, **Credential** token) throws *AuthorizationException*
 - Effect: Register a new model with the MLaaS service.
 - Sequence Diagrams: None
- void setAPIKey(String apiKey) throws *AuthenticationException*
 - Effect: Set the credentials for authenticating with the ML-as-a-Service provider.
 - Sequence Diagrams: None

Diagrams: None

E.3.13 MLaaSService

Provided by: ¶ MLaaS

Required by: ¶ MLModelManager, ¶ MLModelUpdateProcessor, ¶ eHealth Platform

Description : This interface offers operations for interacting with MLaaS service providers.

Operations:

- List<**MLModelUpdate**> fetchModelUpdates(**Credential** credential, **MLModelDescription** model)
 - Effect: Fetch a list of model updates to apply to the local **MLModel**.
 - Returns: The list of MLModelUpdates to apply locally.
 - Sequence Diagrams: fig. D.4
- List<**MLModelDescription**> getAvailableModels(**Credential** credential)
 - Effect: Get the list of available ML models to use for making predictions on incoming sensor data.
 - Returns: The list of available MLModels.
 - Sequence Diagrams: fig. D.3
- void pushModelUpdates(**Credential** credential, **MLModelDescription** model, List<**MLModelUpdate**> updates)
 - Effect: Send a list of MLModelUpdates to the online service.
 - Sequence Diagrams: None
- **MLModel** retrieveModel(**Credential** credential, **MLModelDescription** model)
 - Effect: Retrieve a specific **MLModel** corresponding with the provided **MLModelDescription**.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.3

Diagrams: None

E.3.14 MLAPI

Provided by: ¶ MLLibrary

Required by: ¶ ModelUpdater, ¶ SensorDataRiskAssessment

Description : This is the API provided by the MLLibrary to use **MLModels** locally for making predictions and processing updates to these models.

Operations:

- **MLModel** applyUpdate(**MLModel** model, **MLModelUpdate** update)
 - Effect: Applies the provided **MLModelUpdate** on the provided **MLModel** and returns the resulting **MLModel**.
 - Returns: The resulting **MLModel** after applying the requested **MLModelUpdate**.
 - Sequence Diagrams: fig. D.4
- **Prediction** predict(**SensorDataPackage** sensorData)
 - Effect: Make a prediction with the active **MLModel** on the provided sensordata.

- Returns: The prediction based on the provided sensor data.
- Sequence Diagrams: fig. D.2
- **void verify(MLModel model)** throws *InvalidMLModelException*
 - Effect: Verifies that a provided **MLModel** is working correctly after applying **MLModelUpdates**.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.15 MLModelMgmt

Provided by: MLModelManager, MLModelStorageManager

Required by: MLModelRetrieval, RiskEstimationProcessor

Description : The interface provides operations for managing different **MLModels** locally, querying available models for making predictions on newly incoming sensor data.

Operations:

- **MLModel fetchModel(String modelId)** throws *NoSuchMLModelException*
 - Effect: Fetch the **MLModel** corresponding with the provided modelId.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.2
- **List<String> getAvailableModels()**
 - Effect: Get the list of available **MLModels** that can be used for making **Predictions** locally.
 - Returns: The list of identifiers of the available **MLModels** that can be used.
 - Sequence Diagrams: None
- **void synchronizeModels()**
 - Effect: Synchronize the list of locally-stored **MLModels** by checking for newly-available **MLModels** that are not stored in the local **MLModelRepository** and retrieving these **MLModels** from the **MLaaS** provider.
 - Sequence Diagrams: fig. D.3
- **void updateModel(String modelId)** throws *NoSuchMLModelException*
 - Effect: Update the model with the specified id by fetching **MLModelUpdates** and applying them to the model.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.16 MLModelStorage

Provided by: MLModelRepository

Required by: MLModelStorageManager, MLModelUpdateProcessor

Operations:

- **boolean isAvailable(String modelId)**
 - Effect: Check whether an **MLModel** (identified by the modelId) is available in the local **MLModelRepository**.
 - Returns: Boolean indicating if the specified model is present in the repository.
 - Sequence Diagrams: figs. D.3 and D.4
- **MLModel retrieveModel(String modelId)**
 - Effect: Retrieve the **MLModel** with the specified id.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.4
- **void storeModel(MLModel model)**
 - Effect: Store an **MLModel** in the local **MLModelRepository**
 - Sequence Diagrams: figs. D.3 and D.4

Diagrams: None

E.3.17 MLModelSync

Provided by: MLModelUpdateProcessor

Required by: MLModelStorageManager

Operations:

- **void fetchNewMLModels()**

- Effect: Fetches newly available **MLModels** and store them in the **MLModelRepository**.
- Sequence Diagrams: fig. D.3
- **void processMLModelUpdates(String modelId)** throws *NoSuchMLModelException*
 - Effect: Retrieve and process the updates for the specified **MLModel** and store the updated **MLModel** in the **MLModelRepository**.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.18 MLModelUpdateMgmt

Provided by: **ModelUpdater**

Required by: **MLModelUpdateProcessor**

Operations:

- **MLModel** **processMLModelUpdates(MLModel model, List<MLModelUpdate> updates)**
 - Effect: Requests the **ModelUpdater** to apply the list of **MLModelUpdates** on the provided **MLModel**.
 - Returns: The **MLModel** resulting from the application of the list of **MLModelUpdates** on the initial **MLModel**.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.19 OtherDataMgmt

Provided by: **OtherFunctionality**

Required by: **ClinicalJobCreator**, **ClinicalModelCache**, **RiskEstimationCombiner**, **RiskEstimationScheduler**

Operations:

- **PatientStatus** **getPatientStatus(PatientId patientId)** throws *NoSuchPatientException*
 - Effect: Fetch and return the status of the patient identified by the patientId.
 - Returns: The status of the patient.
 - Sequence Diagrams: None
- **void setEstimatedPatientStatus(PatientId patientId, PatientStatus estimatedStatus, Timestamp estimationTime)** throws *NoSuchPatientException*
 - Effect: Update the patient status estimation of the patient identified by patientId to the given value estimatedStatus and update the time of estimation to estimationTime. The time of estimation is the time at which the corresponding estimation job for this patient was completed. If patient's estimated risk changed the appropriate parties are notified.
 - Sequence Diagrams: figs. D.5 and D.6

Diagrams: None

E.3.20 PatientRecordMgmt

Provided by: **OtherFunctionality**

Required by: **RiskEstimationCombiner**

Operations:

- **PatientRecord** **getPatientRecord(PatientId patientId)** throws *NoSuchPatientRecordException*
 - Effect: This will fetch the EHR record of the patient with given id from the HIS and return it. If the HIS is not available, an older (cached) copy of the patient record is returned if possible.
 - Returns: The requested patient record.
 - Sequence Diagrams: None

Diagrams: None

E.3.21 Results

Provided by: **RiskEstimationCombiner**

Required by: **JobProcessor**, **RiskEstimationProcessor**

Operations:

- **void addClinicalModelJobResults(ClinicalModelJobId jobId, ClinicalModelJobResult results)**

- Effect: Sends the result of the performed clinical model computation (identified by the jobId) to the **RiskEstimationCombiner** for combining with the other partial results belonging to the same risk estimation.
- Sequence Diagrams: figs. D.2 and D.6

Diagrams: None

E.3.22 SensorDataAnalysis

Provided by: `SensorDataRiskAssessment`

Required by: `JobProcessor`

Operations:

- `void performModelPrediction(ClinicalModelJob clinicalModelJob, MLModel model)`
 - Effect: Perform a prediction using the provided **MLModel** and the data provided in the `clinicalModelJob` definition. Afterwards the results are submitted for combination by the **RiskEstimationCombiner**.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.23 SensorDataMgmt

Provided by: `OtherFunctionality`

Required by: `ClinicalJobCreator`, `RiskEstimationScheduler`

Operations:

- `void addSensorData(PatientId patientId, SensorDataPackage sensorData, Timestamp receivedAt)`
 - Effect: Store the given sensor data and meta-data.
 - Sequence Diagrams: None
- `Map<Timestamp, SensorDataPackage> getAllSensorDataOfPatient(PatientId patientId)`
 - Effect: This will fetch and return all sensor data belonging to the patient identified by `patientId`
 - Returns: The sensor data and the timestamp of their arrival in the system.
 - Sequence Diagrams: None
- `Map<Timestamp, SensorDataPackage> getAllSensorDataOfPatientBefore(PatientId patientId, Timestamp stopTime)`
 - Effect: Fetch and return all sensor data belonging to the patient identified by `patientId` which was received before the specified time `stopTime`.
 - Returns: The sensor data and the timestamp of their arrival in the system.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.4 Nodes

E.4.1 eHealthPlatform

Responsibility: Container node representing the deployment context boundary for the eHealth Platform.

Visible on diagrams: fig. C.1

E.4.2 MLaaS Conn (Pilot Deployment)

Responsibility: This node is responsible for the connection with the MLaaS Service.

Visible on diagrams: fig. C.3

E.4.3 MLaaS Service (Dev Deployment)

Responsibility: This represents the MLaaS cloud service.

Visible on diagrams: fig. C.4

E.4.4 MLaaS Service (Pilot Deployment)

Responsibility: This represents the cloud MLaaS service.

Visible on diagrams: fig. C.3

E.4.5 MLaaS Cloud

Responsibility: Cloud infrastructure of the MLaaS Service provider.

Visible on diagrams: fig. C.1

E.4.6 MLaaSConnectorNode

Responsibility: Node responsible for handling connections with external MLaaS providers.

Visible on diagrams: figs. C.1 and C.2

E.4.7 RiskEstimationManagementNode (Pilot Deployment)

Responsibility: The RiskEstimationManagement node hosts the creation, scheduling and combining of the clinical jobs.

Visible on diagrams: fig. C.3

E.4.8 RiskEstimationManagmentNode (Dev Deployment)

Responsibility: The development deployment is a single RiskEstimationManagement node that contains all other functionality, job creation, scheduling, and combination.

Visible on diagrams: fig. C.4

E.4.9 RiskEstimationMgmtNode

Responsibility: The RiskEstimationMgmtNode keeps track of the risk estimation jobs, and schedules and combines the results.

Visible on diagrams: fig. C.2

E.4.10 RiskEstimationProcessorNode

Responsibility: Multiple RiskEstimationProcessor nodes enable the processing of multiple sensorData inputs in parallel.

Visible on diagrams: fig. C.2

E.4.11 RiskEstimationProcessorNode (Dev Deployment)

Responsibility: The development deployment only uses two RiskEstimationProcessorNodes to run **ClinicalModelJobs** in parallel.

Visible on diagrams: fig. C.4

E.4.12 RiskEstimationProcessorNode (Pilot Deployment)

Responsibility: There are five instances of the RiskEstimationProcessorNode to enable the processing of multiple **ClinicalModelJobs** in parallel.

Visible on diagrams: fig. C.3

E.4.13 TODO Node (Pilot Deployment)

Responsibility: This TODO node for the pilot deployment contains the OtherFunctionality of the system that has not been worked out in detail yet.

Visible on diagrams: fig. C.3

E.4.14 TODONode

Responsibility: Temporary node that requires further decomposition.

Visible on diagrams: fig. C.2

E.5 Exceptions

- *AuthenticationException* (Δ *Exception*):
Signals a problem with the credentials used for authentication. These may be invalid or expired.
- *AuthenticationException2* (Δ *SecurityException*):
Missing or invalid credentials.
- *AuthorizationException* (Δ *Exception*):
Signals that the principal has insufficient access rights to perform the operation.
- *AuthorizationException2* (Δ *SecurityException*):
The requestor is not authorized to perform the requested action.
- *Exception*:
Subtypes: Δ *AuthenticationException*, Δ *AuthorizationException*, Δ *InvalidAPIKeyException*, Δ *InvalidMLModelException*, Δ *IOException*, Δ *NoSuchMLModelException*, Δ *NoSuchPatientException*, Δ *NoSuchPatientRecordException*
Base exception class.
- *InvalidAPIKeyException* (Δ *Exception*):
Thrown when no correctly formatted API key is configured for constructing authentication Credential|s.
- *InvalidMLModelException* (Δ *Exception*):
Thrown when a provided MLModel is invalid.
- *IOException* (Δ *Exception*):
Thrown when an IO operation fails.
- *NoSuchMLModelException* (Δ *Exception*):
Thrown when the requested MLModel cannot be found.
- *NoSuchPatientException* (Δ *Exception*):
Thrown if no patient with the specified identifier exists.
- *NoSuchPatientRecordException* (Δ *Exception*):
Thrown if no patient record for the patient with the given identifier exists in the HIS.
- *SecurityException*:
Subtypes: Δ *AuthenticationException2*, Δ *AuthorizationException2*
Parent class for security exceptions.

E.6 Data types

- **Address:**
Attributes: **AddressUse** use, **AddressType** type, String text, List<String> line, String city, String district, String state, String postalCode, String country, **Period** period, long serialVersionUID
Base StructureDefinition for Address Type: An address expressed using postal conventions (as opposed to GPS or other location definition formats). This data type may be used to convey addresses for use in delivering mail as well as for visiting locations which might not be valid for mail delivery. There are a variety of postal address formats defined around the world. The line attribute contains contains the house number, apartment number, street name, street direction, P.O. The text attribute specifies the entire address as it should be displayed e.g. on a postal label. This may be provided instead of or as well as the specific parts.
- **AddressType:**
Enum: POSTAL, PHYSICAL, BOTH, NULL.
Distinguishes between physical addresses (those you can visit) and mailing addresses (e.g. PO Boxes and care-of addresses). Most addresses are both.
- **AddressUse:**

Enum: HOME, WORK, TEMP, OLD, BILLING, NULL.

The purpose of the address.

- **Annotation:**

Attributes: **Date** time, String text, long serialVersionUID, String author

Base StructureDefinition for Annotation Type: A text note which also contains information about who made the statement and when.

- **BigDecimal:**

Immutable, arbitrary-precision signed decimal numbers. A BigDecimal consists of an arbitrary precision integer unscaled value and a 32-bit integer scale

- **ClinicalModel:**

Attributes: String id, String name, String description, List<String> requiredParameters, List<String> optionalParameters, List<String> applicableMLModelIds, boolean requiresHistoricalSensorData

A clinical model. This model has an id, name, description. It contains a list of parameters for using the model and a set of MLModelIds to specify which types of MLModels can be used in the context of the ClinicalModel.

- **ClinicalModelConfiguration:**

Attributes: Map<String, String> modelParameters

The configuration for a clinical model. Represented as a set key/value pair for the parameters of the clinical model.

- **ClinicalModelJob:**

Attributes: **ClinicalModel** clinicalModel, **ClinicalModelJobId** jobId, **SensorDataPackage** sensorData, **PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration, **PatientRecord** patientRecord, Map<**Timestamp**, **SensorDataPackage**> historicalSensorData

A single job for the computation of a clinical model. This contains a ClinicalModelJobId, ClinicalModelId and PatientId respectively identifying the job itself, the corresponding clinical model and the patient. If the corresponding risk estimation the job belongs to is triggered by the arrival of a sensor data package, this sensor data is also contained.

- **ClinicalModelJobId:**

A piece of data uniquely identifying a certain clinical model job in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI etc.

- **ClinicalModelJobResult:**

The result of the computation of a clinical model containing the estimated risk level for the patient. The resulting risk score for ClinicalModels are on the same unit scale so the the results of multiple ClinicalModels can be combined.

- **CodeableConcept:**

Attributes: List<**Coding**> coding, String text, long serialVersionUID

Base StructureDefinition for CodeableConcept Type: A concept that may be defined by a formal reference to a terminology or ontology or may be provided by text.

- **CodeType:**

Attributes: long serialVersionUID, String system, String value

Primitive type "code" in FHIR, when not bound to an enumerated list of codes

- **Coding:**

Attributes: String system, String version, **CodeType** code, String display, boolean userSelected, long serialVersionUID

Base StructureDefinition for Coding Type: A reference to a code defined by a terminology system.

- **ContactComponent:**

Attributes: List<**CodeableConcept**> relationship, String name, List<**ContactPoint**> telecom, **Address** address, String administrativeGender, String organization, **Period** period, long serialVersionUID

A contact party (e.g. guardian, partner, friend) for the patient. The relationship attribute species the nature of the relationship between the patient and the contact person. It is a CodeableConcept (which can be formal reference to terminology or an ontology). For example, in the system <http://hl7.org/fhir/ValueSet/relatedperson-relationshiptype>, it can be coded as CHILD for a child, or MGRMTH for maternal grandmother, etc.

- **ContactPoint:**

Attributes: **ContactPointSystem** system, String value, **ContactPointUse** use, int rank, long serialVersionUID, **Period** period

Base StructureDefinition for ContactPoint Type: Details for all kinds of technology mediated contact points for a person or organization, including telephone, email, etc. The value attribute contains the actual contact point details, in a form that is meaningful to the designated communication system (i.e. phone number or email address). Rank specifies a preferred order in which to use a set of contacts. ContactPoints with lower rank values are more preferred than those with higher rank values. Period specifies the time period when the contact point was/is in use.

- **ContactPointSystem:**

Enum: PHONE, FAX, EMAIL, PAGER, URL, SMS, OTHER, NULL.

Enum denoting the type of communication channel for the ContactPoint.

- **ContactPointUse:**

Enum: HOME, WORK, TEMP, OLD, MOBILE, NULL.

Enum denoting the usage type of the ContactPoint.

- **Credential:**

Authentication object for making authenticated requests to the MLaaS service provider.

- **Date:**

The class Date represents a specific instant in time, with millisecond precision.

- **Identifier:**

Attributes: **IdentifierUse** use, String type, String system, String value, **Period** period, String assigner, long serialVersionUID

Base StructureDefinition for Identifier Type: An identifier - identifies some entity uniquely and unambiguously. Typically this is used for business identifiers. The type is a coded type for the identifier that can be used to determine which identifier to use for a specific purpose. The system attribute establishes the namespace for the value - that is, a URL that describes a set values that are unique. The value attribute specifies the portion of the identifier typically relevant to the user and which is unique within the context of the system. The period specifies the time period during which identifier is/was valid for use. The assigner specifies the organization that issued/manages the identifier.

- **IdentifierUse:**

Enum: USUAL, OFFICIAL, TEMP, SECONDARY, OLD, NULL.

Enum denoting the usage type of a particular Identifier.

- **MLModel:**

Attributes: String modelId, **MLModelDescription** modelDescription

Machine learning model for making predictions based on sensordata. The actual internal encoding of the model itself is left open.

- **MLModelDescription:**

Attributes: String uuid, String name, String description, String providerName

Object providing the MLModel meta data. It does not contain the model itself.

- **MLModelUpdate:**

Attributes: **MLModelDescription** modelDescription

Represents a model update that can be applied on an MLModel. These model updates enable exchanging changes to the MLModel between different parties without sending the actual MLModel to the other parties.

- **Object:**

Root class for objects that can be stored in the Key-Value storage service. Data types that need to be saved need to inherit from this class.

- **Observation:**

Attributes: List<**Identifier**> identifier, List<**Identifier**> partOf, **ObservationStatus** status, List<**CodeableConcept**> category, **CodeableConcept** code, **Patient** subject, **Period** effective, **Date** issued, List<**Identifier**> performer, String value, **CodeableConcept** dataAbsentReason, **CodeableConcept** interpretation, List<**Annotation**> note, **CodeableConcept** bodySite, **CodeableConcept** method, String device, List<**Observation**> hasMember, List<**Observation**> derivedFrom, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange, **ObservationComponentComponent** component

Measurements and simple assertions made about a patient, device or other subject. The partOf attribute specifies the larger event of which this particular Observation is a component

or step. For example, an observation as part of a procedure.

The category specifies a code that classifies the general type of observation being made. For example, in the system <http://terminology.hl7.org/CodeSystem/observation-category>, vital-signs is used for clinical observations such as blood pressure, heart rate, etc.

The code attribute describes what was observed. Sometimes this is called the observation 'name'. For example, in the system <http://loinc.org>, code '8867-4' is used for expressing the heart rate, '85354-9' for a blood pressure panel (which must have components '8480-6' and '8462-4' for, respectively, systolic and diastolic pressure).

The performer is who was responsible for asserting the observed value as 'true'. This can refer to a Patient, Practitioner, organization, etc. The value is the information determined as a result of making the observation, if the information has a simple value.

The dataAbsentReason provides a reason why the expected value in the element Observation.value[x] is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation is a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

The bodySite indicates the site on the subject's body where the observation was made (i.e. the target site). For example, in the system <http://snomed.info/sct>, code '344001' indicates the ankle.

The method indicates the mechanism used to perform the observation. For example, in the system <http://snomed.info/sct>, code '46973005' indicates blood pressure taking procedure.

The device attribute specifies an identifier of the device used to generate the observation data.

- **ObservationComponentComponent:**

Attributes: String code, **Range** value, String dataAbsentReason, List<String> interpretation, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange

Some observations have multiple component observations. These component observations are expressed as separate code value pairs that share the same attributes. Examples include systolic and diastolic component observations for blood pressure measurement and multiple component observations for genetics observations. The code describes what was observed. Sometimes this is called the observation 'code'. For example, in the system <http://loinc.org>, '8480-6' is used for the systolic component of the blood pressure.,

The dataAbsentReason provides a reason why the expected value in the element Observation.component.value[x] is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation provides a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

- **ObservationReferenceRangeComponent:**

Attributes: **Quantity** low, **Quantity** high, **CodeableConcept** type, List<**CodeableConcept**> appliesTo, **Range** age, String text, long serialVersionUID

Guidance on how to interpret the value by comparison to a normal or recommended range. Multiple reference ranges are interpreted as an "OR". In other words, to represent two distinct target populations, two 'referenceRange' elements would be used.

The appliesTo specifies codes to indicate the target population this reference range applies to. For example, in the system <http://snomed.info/sct>, code '248152002' indicates female.

The type specifies codes to indicate the what part of the targeted reference population it applies to. For example, in the system <http://terminology.hl7.org/CodeSystem/referencrange-meaning>, 'normal' for the 95% normal range, or 'recommended' for the recommended range by a relevant professional body.

The text attribute contains a text based reference range in an observation which may be used when a quantitative range is not appropriate for an observation.

- **ObservationStatus:**

Enum: REGISTERED, PRELIMINARY, FINAL, AMENDED, CORRECTED, CANCELLED, ENTEREDINERROR, UNKNOWN, NULL.

The status of the observation result value or the RiskAssessment.

- **Patient:**

Attributes: List<**Identifier**> identifier, boolean active, List<String> name, String

administrativeGender, **Date** birthDate, **Date** deceased, List<**Address**> address, boolean multipleBirth, List<**Identifier**> generalPractitioner, **Identifier** managingOrganization, long serialVersionUID, **ContactComponent** contact

Demographics and other administrative information about an individual or animal receiving care or other health-related services.

- **PatientId:**

A piece of data uniquely identifying a patient in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI, etc.

- **PatientRecord:**

The structured contents of the EHR record of a certain patient, including medication history, recent treatments, allergies, etc.

- **PatientStatus:**

The status of a patient, i.e., their risk level.

- **Period:**

Attributes: **Date** start, **Date** end, long serialVersionUID

Base StructureDefinition for Period Type: A time period defined by a start and end date and optionally time.

- **Prediction:**

A prediction from the MLModel.

- **Quantity:**

Attributes: **BigDecimal** value, String unit, String system, **CodeType** code, long serialVersionUID

Base StructureDefinition for Quantity Type: A measured amount (or an amount that can potentially be measured). Note that measured amounts include amounts that are not precisely quantified, including amounts involving arbitrary units and floating currencies. Unit specifies a human-readable form of the unit. System contains the identification of the system that provides the coded form of the unit. The code contains a computer processable form of the unit in some unit representation system. For example, blood pressure can be expressed in the system <http://unitsofmeasure.org> and the unit mm[Hg].

- **Query<T>:**

Attributes: List<T> values, List<String> qualifier

The Query object is used to express search restrictions in a string-based format for interacting with the REST-based health API. For example, before a specified date is encoded as 'ltyyyy-MM-dd'. The list of qualifiers specify how the corresponding value should be used (e.g., lt for less than a date, matches if it should be present in a string, exact for an exact string match).

- **Range:**

Attributes: long serialVersionUID, **Quantity** low, **Quantity** high

Base StructureDefinition for Range Type: A set of ordered Quantities defined by a low and high limit.

- **RiskAssessment:**

Attributes: List<**Identifier**> identifier, **ObservationStatus** status, **CodeableConcept** method, **CodeableConcept** code, **Patient** subject, **Date** occurrence, String condition, **Identifier** performer, List<**CodeableConcept**> reason, List<**Observation**> basis, String mitigation, List<**Annotation**> note, long serialVersionUID, **RiskAssessmentPredictionComponent** prediction

An assessment of the likely outcome(s) for a patient or other subject as well as the likelihood of each outcome. Method specifies the algorithm, process or mechanism used to evaluate the risk. Code specifies the type of the risk assessment performed. For example, in the system <http://snomed.info/sct>, '709510001' represents Assessment of risk for disease (procedure). For assessments or prognosis specific to a particular condition, the condition attribute indicates the condition being assessed. It should be a reference to an existing condition but is a simplified as a string here. Reason specifies the reason the risk assessment was performed. Mitigation contains a description of the steps that might be taken to reduce the identified risk(s).

- **RiskAssessmentPredictionComponent:**

Attributes: String outcome, **Range** probability, String qualitativeRisk, **BigDecimal** relativeRisk, **Date** when, String rationale, long serialVersionUID

Describes the expected outcome for the subject. Outcome specifies one of the potential outcomes for the patient (e.g. remission, death, a particular condition). QualitativeRisk

indicates how likely the outcome is (in the specified timeframe), expressed as a qualitative value (e.g. low, medium, or high). Rationale contains additional information explaining the basis for the prediction. Both outcome and qualitativeRisk are CodeableConcepts (which can be formal reference to terminology or an ontology), but simplified as a string.

- **RiskEstimationId:**

A piece of data uniquely identifying a risk estimation performed by the PMS. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URL etc.

- **SensorDataElement:**

Attributes: String sensorId, String sensorType, String measurementType, String measurementUnit, **BigDecimal** measurementValue, String measurementValueString

Each sub-package lists the id of the sensor, the type of the sensor, the type of the measurements and the measurements itself. These measurements range from a single value (e.g., in case of the blood pressure) to a complex data structure (e.g., in case of an ECG). Non-numerical types of measurements can be captured in the measurementValueString attribute.

- **SensorDataPackage:**

Attributes: List<**SensorDataElement**> sensorDataElements

A package of sensor data, i.e., a list of sensorDataElements.

- **Timestamp:**

The representation of a time (i.e., date and time of the day) in the system.

E.7 Unresolved issues

SA Plugin v9.0.0 (VP OpenAPI v17.2)

- [Comp.06] : 1 (*Component x has no visualizations*)
- [Diag.comp.06] : 1 (*The component x may be missing on the C/S context diagram.*)
- [Diag.depl.03] : 1 (*Component x or its sub-components have not been deployed*)
- [If.03] : 1 (*The interface x is not required by any component or module*)